VECTOR

Static Code Analysis by Vector

Version 2.0
2023-01-25
Application Note AN-ISC-8-1135

Author Hörner, Hartmut

Restrictions Customer Confidential

Abstract Static code analysis based on MISRA and AUTOSAR is a mandatory part of the development process applied for embedded software components. This application note describes the static code analysis performed by Vector.

Table of Contents

1.0 Overview 1
1.1 Static Code Analysis Performed by Vector 1
1.2 Impact of Configuration 2
1.3 Impact of Analysis Tool 2
1.4 Other Programming Standards 2
1.5 Other Static Code Analysis Tools 2
2 Additional Resources 3
3 Contacts 4

1.0 Overview

Static code analysis is widely used during the development of embedded software to verify the absence of specific kinds of errors and to check the adherence to programming standards. A variety of tools and standards exists. In automotive projects typically the MISRA-C and AUTOSAR C++ guidelines are applied.

Static code analysis based on those guidelines is a mandatory part of the development process applied for embedded software components by Vector. One of the development goals is to reduce the number of MISRA-C and AUTOSAR C++ warnings.

However, if such analysis is performed by the user warnings may still be present for several reasons. This is analyzed in the following sections.

1.1 Static Code Analysis Performed by Vector

For embedded software components written in the C language a MISRA-C:2012 analysis is made with the tool QAC. Compliance with the AUTOSAR C++ guideline is checked with the tool Bauhaus. Since the software components are typically configurable the analysis has to be performed for a component-specific set of configurations. Analysis results are documented in the static analysis report and deviations are justified. Product-specific deviation documents exist for rules where deviations are expected and cannot be avoided (e.g. because a specific programming style is required by another specification).

Static analysis is part of the component development process; it is not repeated during Vector's integration test.

Other static analysis methods or tools are not applied by default. There may be deviations from this rule if explicitly agreed with the customer for a specific project.

On request the product-specific deviation document, e.g. for CANbedded or MICROSAR products documents can be provided.

1.2 Impact of Configuration

As already mentioned, the result of the static analysis depends on the configuration of the software component. When static analysis is performed by the user the configuration may be different and it can happen that the analysis result is not identical.

There are two options how this issue can be addressed:

1) To reproduce the static check result documented by Vector use the same or at least a very similar configuration. Information about the configuration used during Vector's internal tests can be provided on request.
2) If static analysis shall be repeated for a specific user defined configuration this can be quoted as additional service.

1.3 Impact of Analysis Tool

When the same code is analyzed not all analysis tools deliver compatible results. Therefore, the following pre-requisites exist to reproduce a static analysis result:

1) same tool
2) same tool version
3) same tool configuration (e.g. compatible set of excluded checking rules)

Information about the tool version and configuration used by Vector can be provided on request.

If you send us results generated by another tool, we may not be able to reproduce them if the tool is not available at Vector. Please note, that we can analyze such tool output only if there is a clear indication by the tool that a functional issue was detected in Vector's embedded software.

1.4 Other Programming Standards

Only compliance with the PES guidelines for C and C++ which are based on MISRA-C:2012 and AUTOSAR C++ 17-10 is checked. Other versions of of the underlying standards are out of scope. In the MISRA and AUTOSAR guideline specifications mapping tables between the different versions are provided.

The Vector coding style guide serves as basis for the implementation. Other customer-specific coding style guides are not considered.

1.5 Other Static Code Analysis Tools

The tools QAC (for C) and Bauhaus (for C++) are the only mandatory static analyzers in Vector's development process for embedded software. More tools are available which are used *optionally* for special use cases or if project specific agreements exist.

- PC-Lint and clang-tidy can be used for static code analysis both during component development and integration. The rule checking capability of this tool is regarded as a subset of QAC and Bauhaus.
- Understand C can be used for interactive code analysis and special kinds of analysis.

Note that the usage of these tools is optional and therefore no quality reports exist unless other project specific agreements exist.

Copyright © 2023 - Vector Informatik GmbH

Contact Information: www.vector.com or +49-711-80 670-0

For modules with an ASIL level additional internal tools such as the VectorCodeAnalyzer are used to support freedom from interference analysis.

If reports from other tools than the ones listed above are submitted to Vector, we may not be able to analyze them depending on the quality of the report. It may be necessary that the tool is provided to Vector to reproduce the findings. Please note, that such analysis is only performed if there is a clear indication by the tool that a functional issue was detected in Vector's embedded software.

2 Additional Resources

MISRA C:2012 Guidelines for the use of the C language in critical systems

AUTOSAR 17-10 Guidelines for the use of the C++14 language in critical and safety-related systems

For more information about the above-mentioned analysis tools please refer to the homepages of the tool vendors.

Copyright © 2023 - Vector Informatik GmbH

Contact Information: www.vector.com or +49-711-80 670-0

3 Contacts

For a full list with all Vector locations and addresses worldwide, please visit http://vector.com/contact/.

Copyright © 2023 - Vector Informatik GmbH